

# Java Coding Guidelines

Writing clean, readable and professional code — a beginner's handbook

---

For engineering graduates starting their journey as software developers. Every rule is shown with a BAD example, a GOOD example, and the reason behind it — covering naming, formatting, comments, methods, classes, types, error handling and edge cases.

Version 1.0 · September 2026

Target: Java 17 (applies to Java 8 and above)

# Contents

Use this index to jump to any topic. Each numbered chapter is a self-contained lesson; sections 13 and 14 are quick references you will return to often.

|                                                                 |    |
|-----------------------------------------------------------------|----|
| 1. Why Coding Guidelines Matter.....                            | 3  |
| 1.1 What "clean code" means.....                                | 3  |
| 1.2 How to use this document.....                               | 4  |
| 1.3 The industry standard we follow.....                        | 4  |
| 2. Naming: The Foundation of Readable Code.....                 | 4  |
| 2.1 The five rules for any name.....                            | 4  |
| 2.2 Building names from known words and objectives.....         | 5  |
| 2.3 Variables — camelCase.....                                  | 5  |
| 2.4 Constants — UPPER_SNAKE_CASE.....                           | 6  |
| 2.5 Methods — verbs in camelCase.....                           | 6  |
| 2.6 Classes and interfaces — PascalCase nouns.....              | 7  |
| 2.7 Packages — all lowercase.....                               | 8  |
| 2.8 Naming summary table.....                                   | 8  |
| 3. Formatting: Indentation, Braces, Spacing and Layout.....     | 9  |
| 3.1 Indentation — 4 spaces, never tabs.....                     | 9  |
| 3.2 Braces — K&R style, and always use them.....                | 9  |
| 3.3 Spacing — let the code breathe.....                         | 10 |
| 3.4 Line length — 100 characters maximum.....                   | 11 |
| 3.5 Vertical spacing — blank lines group related ideas.....     | 11 |
| 3.6 One statement per line, one variable per declaration.....   | 12 |
| 3.7 Order of members inside a class.....                        | 13 |
| 3.8 Imports.....                                                | 13 |
| 4. Comments and Documentation.....                              | 14 |
| 4.1 Comment the WHY, not the WHAT.....                          | 14 |
| 4.2 Javadoc for anything public.....                            | 14 |
| 4.3 Comments that should never exist.....                       | 15 |
| 4.4 Self-documenting code beats comments.....                   | 15 |
| 5. Writing Good Methods.....                                    | 16 |
| 5.1 Small, and doing one thing.....                             | 16 |
| 5.2 Keep parameters few.....                                    | 17 |
| 5.3 Guard clauses — return early, avoid deep nesting.....       | 18 |
| 5.4 No magic numbers or magic strings.....                      | 18 |
| 5.5 Avoid boolean flag parameters.....                          | 19 |
| 5.6 Return what the name promises; no side effects.....         | 19 |
| 6. Writing Good Classes.....                                    | 20 |
| 6.1 One responsibility per class.....                           | 20 |
| 6.2 Encapsulation — fields are private.....                     | 21 |
| 6.3 Prefer immutability and final.....                          | 21 |
| 6.4 Constructors should produce a valid object.....             | 22 |
| 6.5 Always override equals, hashCode and toString together..... | 22 |
| 7. Types, Variables and Declarations.....                       | 22 |
| 7.1 Choose the right type.....                                  | 22 |
| 7.2 Declare close to first use, and initialise.....             | 23 |
| 7.3 Program to the interface.....                               | 24 |
| 7.4 Using var sensibly.....                                     | 24 |
| 7.5 Strings — the three classic mistakes.....                   | 24 |
| 8. Control Flow: Conditions, Loops and Switch.....              | 25 |
| 8.1 Simplify boolean expressions.....                           | 25 |
| 8.2 Name complex conditions.....                                | 25 |
| 8.3 Loops — prefer the enhanced for loop and streams.....       | 26 |

|                                                                        |    |
|------------------------------------------------------------------------|----|
| 8.4 Switch — use the modern arrow form.....                            | 26 |
| 8.5 Avoid long if-else chains on type or status.....                   | 27 |
| 9. Error Handling.....                                                 | 27 |
| 9.1 Exceptions in 60 seconds.....                                      | 27 |
| 9.2 Never swallow an exception.....                                    | 28 |
| 9.3 Catch the most specific exception, not Exception or Throwable..... | 28 |
| 9.4 Fail fast: validate inputs at the boundary.....                    | 29 |
| 9.5 Exception messages must be useful.....                             | 29 |
| 9.6 try-with-resources for anything that must be closed.....           | 30 |
| 9.7 Custom exceptions: name the business problem.....                  | 30 |
| 9.8 Do not use exceptions for normal control flow.....                 | 30 |
| 9.9 Logging instead of System.out.....                                 | 31 |
| 10. Edge Case Handling.....                                            | 31 |
| 10.1 The edge-case checklist.....                                      | 32 |
| 10.2 Null — the billion-dollar mistake.....                            | 32 |
| 10.3 Empty and blank input.....                                        | 33 |
| 10.4 Boundaries and off-by-one errors.....                             | 33 |
| 10.5 Integer overflow and division.....                                | 34 |
| 10.6 Floating point comparison.....                                    | 34 |
| 10.7 Modifying a collection while iterating.....                       | 34 |
| 10.8 Untrusted input from users and files.....                         | 35 |
| 11. Code Length, Duplication and Structure.....                        | 35 |
| 11.1 How long is too long?.....                                        | 35 |
| 11.2 Don't Repeat Yourself (DRY).....                                  | 36 |
| 11.3 Delete dead code.....                                             | 36 |
| 11.4 Keep related things together.....                                 | 37 |
| 12. Putting It All Together: A Complete Before-and-After.....          | 37 |
| 13. Self-Review Checklist.....                                         | 39 |
| Naming.....                                                            | 39 |
| Formatting.....                                                        | 40 |
| Methods and classes.....                                               | 40 |
| Comments.....                                                          | 40 |
| Error handling.....                                                    | 40 |
| Edge cases.....                                                        | 40 |
| 14. Appendix: Tools and Further Reading.....                           | 41 |
| 14.1 Let the tools enforce the rules.....                              | 41 |
| 14.2 Recommended reading.....                                          | 41 |
| 14.3 A note on interviews.....                                         | 41 |

# 1. Why Coding Guidelines Matter

You already know how to make a program **work**. This document is about the next step: making a program that other people (and you, six months from now) can **read, trust, and change** without fear.

In college, code is usually written once, run once, and submitted. In a company, code is written once and **read hundreds of times** — by teammates, reviewers, testers, and the on-call engineer at 2 AM trying to fix a production bug. Studies consistently find that developers spend far more time reading code than writing it. That is why professionals care so much about how code looks, not just what it does.

## 1.1 What "clean code" means

Clean code is code that a reader can understand **on the first pass**, without needing the author beside them to explain. It has four qualities:

- **Readable** — names, spacing and structure tell the story. You should be able to read a method almost like a paragraph of English.
- **Predictable** — it does what its name says, and nothing surprising. No hidden side effects.
- **Robust** — it handles bad input, empty input, and failures gracefully instead of crashing with a stack trace.
- **Consistent** — every file in the project looks like it was written by the same careful person, even if ten people wrote it.

## 1.2 How to use this document

Each section explains one topic, shows a **BAD** example (red) and a **GOOD** example (green), and explains exactly what changed and why. Sections end with short practice exercises. The last two chapters contain a full before-and-after program and a self-review checklist that you should run through before submitting any assignment or raising a pull request.

The examples target **Java 17** (the current long-term-support version used by most companies), but almost everything applies to Java 8 onwards.

### A rule to remember

If you have to explain your code to someone, that is a sign the code should be rewritten so that it explains itself. Comments are the last resort, not the first.

## 1.3 The industry standard we follow

These guidelines are aligned with the **Google Java Style Guide** and the **Oracle Java Code Conventions**, which most Indian and global IT companies use as their baseline. Learning them now means you will not have to unlearn habits in your first job. Where the two guides differ, we follow Google (for example, 4-space indentation is used here because it is what most IDEs produce by default and what students see in textbooks; Google itself uses 2 — either is fine as long as the whole project uses one).

# 2. Naming: The Foundation of Readable Code

Naming is the single most important skill in this document. A good name removes the need for a comment. A bad name forces every reader to open the code and reverse-engineer what a thing does. You will name hundreds of things per day as a developer, so this habit pays back enormously.

## 2.1 The five rules for any name

1. **Reveal intent.** The name answers: why does it exist, what does it hold, how is it used?  
`elapsedTimeInDays` answers all three; `d` answers none.
2. **Use real, known words.** Prefer plain English that a teammate from any background recognises. No invented abbreviations (`cstmr`, `calcAmt`, `svcMgr`). Standard ones are fine: `id`, `url`, `html`, `max`, `min`.
3. **Be pronounceable and searchable.** If you cannot say it aloud in a code review (`genymdhms`), rename it (`generationTimestamp`). If you cannot search for it (`e`, `x1`), it will be impossible to refactor later.
4. **One concept, one word.** Do not mix `fetch`, `retrieve` and `get` for the same idea across the project. Pick one and use it everywhere.
5. **Match the scope to the length.** A loop counter that lives for two lines can be `i`. A field that lives for the whole class needs a full descriptive name. The longer something lives, the longer its name should be.

## 2.2 Building names from known words and objectives

A strong name is usually built from two components: a **known word** (a real English noun or verb that anyone on the team recognises) and an **objective** (what it is for, or what it holds). Think of it as *\*what\* + \*purpose\**.

| Bad name | Known word | Objective             | Good name             |
|----------|------------|-----------------------|-----------------------|
| d        | days       | since last login      | daysSinceLastLogin    |
| list1    | students   | who are active        | activeStudents        |
| tmp      | total      | of the order          | orderTotal            |
| flag     | is         | the account verified? | isAccountVerified     |
| data     | response   | from the payment API  | paymentResponse       |
| calc()   | calculate  | monthly EMI           | calculateMonthlyEmi() |

Notice that the good names are longer. **Length is not a problem; vagueness is.** Your IDE auto-completes names, so typing cost is zero. Reading cost is what matters.

### BAD — names that force the reader to guess

```
int d;           // days? distance? diameter?
String s;        // what string?
List<String> list1; // list of what?
boolean flag;    // flag for what?
double val, val2; // meaningless
Object data;     // everything is "data"
```

### GOOD — names that read as English

```
int daysSinceLastLogin;
String customerEmail;
List<String> pendingOrderIds;
boolean isEmailVerified;
double basePrice, discountedPrice;
PaymentResponse paymentResponse;
```

## 2.3 Variables — camelCase

Local variables, fields (instance variables) and method parameters use **lowerCamelCase**: start with a lowercase letter, then capitalise the first letter of each following word. No underscores, no Hungarian prefixes (`strName`, `iCount`), no type in the name.

- **Nouns** for values: `studentName`, `totalAmount`, `retryCount`.
- **Plural nouns** for collections: `students`, `orderIds`, `pricesByProduct` (a map reads well as *\*valueByKey\**).
- **Booleans read as a question**: prefix with `is`, `has`, `can`, `should`: `isActive`, `hasDiscount`, `canRetry`. Never `notActive` or `flag`.
- **Single letters only for tiny scopes**: `i`, `j` as loop counters, `e` in a catch block, `x/y` for coordinates. Anywhere else, spell it out.
- **Avoid number suffixes**: `student1`, `student2` almost always means you need a list or two meaningful names (`currentStudent`, `previousStudent`).

**BAD**

```
String Student_Name = "Asha";
int Age = 21;
boolean active_flag = true;
ArrayList<Integer> marksArrayList = new ArrayList<>();
String str = "9876543210";
int n1 = 10, n2 = 20;
```

**GOOD**

```
String studentName = "Asha";
int age = 21;
boolean isActive = true;
List<Integer> marks = new ArrayList<>();
String phoneNumber = "9876543210";
int minMarks = 10, maxMarks = 20;
```

## 2.4 Constants — UPPER\_SNAKE\_CASE

A constant is a `static final` field whose value never changes. Constants use all capitals with underscores between words. The name should describe the **meaning**, not the value.

**BAD**

```
static final int TEN = 10;           // describes the value, not the meaning
static final double pi = 3.14159;   // wrong case
static final int maxusers = 100;     // unreadable
if (attempts > 3) { ... }            // "magic number" - what is 3?
```

**GOOD**

```
static final int MAX_LOGIN_ATTEMPTS = 3;
static final double GST_RATE = 0.18;
static final int MAX_USERS_PER_BATCH = 100;
static final String DEFAULT_COUNTRY_CODE = "+91";

if (attempts > MAX_LOGIN_ATTEMPTS) { ... }
```

## 2.5 Methods — verbs in camelCase

A method **does** something, so its name starts with a **verb**, followed by the object it acts on. The name should be specific enough that a reader knows what it returns and whether it changes anything.

| Purpose                    | Verb to use                                                                 | Examples                                                                                               |
|----------------------------|-----------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| Compute and return a value | calculate, compute, count, find                                             | <code>calculateTax()</code> ,<br><code>countActiveUsers()</code> ,<br><code>findStudentById(id)</code> |
| Read a field               | <code>get</code>                                                            | <code>getName()</code> , <code>getBalance()</code>                                                     |
| Change a field             | <code>set</code> , <code>update</code>                                      | <code>setName(name)</code> ,<br><code>updateStatus(status)</code>                                      |
| Answer yes/no              | <code>is</code> , <code>has</code> , <code>can</code> , <code>should</code> | <code>isEmpty()</code> , <code>hasPermission()</code> ,<br><code>canWithdraw(amount)</code>            |

| Purpose          | Verb to use               | Examples                                                                                   |
|------------------|---------------------------|--------------------------------------------------------------------------------------------|
| Create something | create, build, generate   | <code>createInvoice()</code> , <code>generateOtp()</code>                                  |
| Convert          | to, parse, format         | <code>toJson()</code> , <code>parseDate(text)</code> , <code>formatCurrency(amount)</code> |
| Check and throw  | validate, ensure, require | <code>validateEmail(email)</code> , <code>ensureNotNull(obj)</code>                        |
| Persist / send   | save, delete, send, load  | <code>saveOrder(order)</code> , <code>sendWelcomeEmail(user)</code>                        |

**BAD**

```
void data() { ... }           // not a verb; does what?
int process(int x) { ... }    // "process" says nothing
boolean check(User u) { ... } // check what?
void doIt() { ... }
int stud(int id) { ... }      // abbreviation + not a verb
void handleStuff() { ... }
double calc(double a, double b) { ... }
```

**GOOD**

```
void loadStudentRecords() { ... }
int calculateFinalScore(int marks) { ... }
boolean isEligibleForPlacement(User user) { ... }
void sendReminderEmail() { ... }
Student findStudentById(int studentId) { ... }
void archiveCompletedOrders() { ... }
double calculateSimpleInterest(double principal, double rate) { ... }
```

**The "surprise" test for method names**

A method named `getUser()` must **only** return a user. If it also creates one when missing, or writes to a log file, or increments a counter, it is lying. Readers will be surprised, and surprised readers introduce bugs.

Either name it honestly (`getOrCreateUser()`) or split it into two methods.

## 2.6 Classes and interfaces — PascalCase nouns

A class represents a **thing**, so its name is a **noun** or noun phrase in **UpperCamelCase** (PascalCase). Every word starts with a capital letter, including the first. Avoid catch-all words such as `Manager`, `Helper`, `Util`, `Data`, `Info`, `Processor` — they hide what the class really does and turn into dumping grounds.

**BAD**

```
class studentdata { ... }    // wrong case, "data" is vague
class Student_Manager { ... } // underscore, "Manager" is vague
class DoPayment { ... }      // a verb - classes are nouns
class Utils { ... }          // everything ends up here
class Info { ... }
interface IStudentRepository { ... } // "I" prefix is C# style, not Java
```

**GOOD**

```

class Student { ... }
class StudentGradeCalculator { ... }
class PaymentGateway { ... }
class DateFormatter { ... }
class EnrollmentRequest { ... }
interface StudentRepository { ... }
class DatabaseStudentRepository implements StudentRepository { ... }

```

- **Exceptions** end with **Exception**: `InsufficientBalanceException`.
- **Interfaces** are plain nouns or adjectives: `Comparable`, `Runnable`, `PaymentProcessor`. Implementations add a qualifier: `RazorpayPaymentProcessor`.
- **Test classes** mirror the class under test: `StudentGradeCalculatorTest`.
- **Enums** are singular nouns; their values are constants: `enum OrderStatus { PENDING, SHIPPED, DELIVERED }`.

## 2.7 Packages — all lowercase

Package names are all lowercase, with no underscores, and follow the reversed domain of the organisation, then the feature: `com.algorithms365.assessment.grading`. Group by **feature** (student, payment, report), not by type (controllers, models, utils) — feature packages keep related code together.

## 2.8 Naming summary table

| Element               | Convention                | Good example                                                      | Bad example                                                  |
|-----------------------|---------------------------|-------------------------------------------------------------------|--------------------------------------------------------------|
| Package               | all lowercase             | <code>com.algo.billing</code>                                     | <code>com.Algo.Billing_Module</code>                         |
| Class / Enum / Record | PascalCase noun           | <code>InvoiceGenerator</code>                                     | <code>invoice_gen</code> , <code>Utils</code>                |
| Interface             | PascalCase noun/adjective | <code>Payable</code> , <code>UserRepository</code>                | <code>IUserRepo</code>                                       |
| Method                | camelCase verb phrase     | <code>calculateTotal()</code>                                     | <code>Total()</code> , <code>doCalc()</code>                 |
| Variable / parameter  | camelCase noun            | <code>unitPrice</code>                                            | <code>UnitPrice</code> , <code>up</code> , <code>p1</code>   |
| Boolean               | is/has/can + camelCase    | <code>isPaid</code> , <code>hasAccess</code>                      | <code>paid</code> , <code>flag</code> , <code>notPaid</code> |
| Constant              | UPPER_SNAKE_CASE          | <code>MAX_RETRY_COUNT</code>                                      | <code>maxRetryCount</code> , <code>THREE</code>              |
| Collection            | plural noun               | <code>orders</code> , <code>namesById</code>                      | <code>orderList</code> , <code>orderArr</code>               |
| Type parameter        | single capital            | <code>T</code> , <code>K</code> , <code>V</code> , <code>E</code> | <code>type</code> , <code>Element</code>                     |

### Practice exercises

1. Rename each of these so a stranger understands them: `int x1`; (marks in maths), `String s`; (a college roll number), `boolean b`; (whether the student has paid fees), `List<Integer> arr`; (attendance percentages), `double t`; (temperature in Celsius).
2. These method names break the rules. Fix them: `void student()`, `int data(int a)`, `boolean checkIt(Order o)`, `void Print_Report()`, `String get(String k)`.
3. Fix the class names: `class userManager`, `class DoLogin`, `class Helper2`, `class student_details`, `interface IShape`.
4. Write a class name, three field names, and two method names for a "library book that can be borrowed

and returned". Use the known-word + objective pattern.

## 3. Formatting: Indentation, Braces, Spacing and Layout

Formatting is about **visual structure**. The compiler does not care about whitespace, but human eyes rely on it completely. Well-formatted code lets a reader see the shape of the logic — which lines belong to which `if`, where a method starts and ends — before reading a single word. The good news: your IDE can do most of this for you (in IntelliJ IDEA press `Ctrl+Alt+L`, in Eclipse `Ctrl+Shift+F`, in VS Code `Shift+Alt+F`). Learn the rules so you can recognise when the formatter is right.

### 3.1 Indentation — 4 spaces, never tabs

Every time you open a block (`{`), indent the contents by **4 spaces**. Every time you close one (`}`), go back 4 spaces. Configure your editor to insert spaces when you press `Tab`, so files look identical on every machine. Mixed tabs and spaces are the number one cause of "it looks fine on my laptop but broken in the review tool".

#### BAD — no indentation; the structure is invisible

```
public class Bank {
public void withdraw(double amount) {
if (amount > balance) {
System.out.println("Insufficient balance");
} else {
balance = balance - amount;
}
}
}
```

#### GOOD — each block steps in by 4 spaces

```
public class Bank {

    public void withdraw(double amount) {
        if (amount > balance) {
            System.out.println("Insufficient balance");
        } else {
            balance = balance - amount;
        }
    }

}
```

### 3.2 Braces — K&R style, and always use them

Java uses the "Egyptian" / K&R brace style: the opening brace goes at the **end of the same line**, and the closing brace goes on its **own line**, aligned with the start of the statement. Never put the opening brace on a new line (that is C# style).

Also, **always use braces**, even for a single statement. Skipping braces saves one line and has caused famous real-world security bugs (Apple's "goto fail" bug in 2014 was exactly this).

**BAD — missing braces create invisible bugs**

```

if (isValid)
    process(order);
    sendEmail(order);    // looks inside the if, but is NOT - always runs

for (int i = 0; i < n; i++) total += i;

public void save()
{
    // C# style brace placement
}

```

**GOOD**

```

if (isValid) {
    process(order);
    sendEmail(order);
}

for (int i = 0; i < n; i++) {
    total += i;
}

public void save() {
    // brace on the same line as the declaration
}

```

### 3.3 Spacing — let the code breathe

Horizontal spacing follows the same rules as English punctuation: a space **after** a comma, a space **around** binary operators, and a space between a keyword and its bracket. No space between a method name and its (.

| Where             | Rule                    | Correct                                     | Wrong                                     |
|-------------------|-------------------------|---------------------------------------------|-------------------------------------------|
| Binary operators  | space on both sides     | <code>a + b, x == y, i &lt;= n</code>       | <code>a+b, x==y</code>                    |
| Assignment        | space on both sides     | <code>total = 0;</code>                     | <code>total=0;</code>                     |
| Commas            | space after, not before | <code>add(a, b, c)</code>                   | <code>add(a,b,c), add(a , b)</code>       |
| Keywords          | space before (          | <code>if (x), while (y), for (...)</code>   | <code>if(x), while(y)</code>              |
| Method calls      | no space before (       | <code>print(x)</code>                       | <code>print (x)</code>                    |
| Unary operators   | no space                | <code>-x, !done, i++</code>                 | <code>- x, ! done</code>                  |
| Casts             | space after )           | <code>(int) value</code>                    | <code>(int)value</code>                   |
| Generics          | no spaces inside <>     | <code>Map&lt;String, Integer&gt;</code>     | <code>Map&lt; String, Integer &gt;</code> |
| Semicolons in for | space after each        | <code>for (int i = 0; i &lt; n; i++)</code> | <code>for(int i=0;i&lt;n;i++)</code>      |

**BAD**

```
for(int i=0;i<marks.length;i++){
```

**BAD**

```

    if(marks[i]>=passMark&&marks[i]<=100){
        total+=marks[i];
        count ++ ;
    }
}
double avg=(double)total/count;
System.out.println( "Average: "+avg );

```

**GOOD**

```

for (int i = 0; i < marks.length; i++) {
    if (marks[i] >= passMark && marks[i] <= 100) {
        total += marks[i];
        count++;
    }
}
double average = (double) total / count;
System.out.println("Average: " + average);

```

### 3.4 Line length — 100 characters maximum

Keep every line under **100 characters** (Google standard; some teams use 80 or 120). Long lines force horizontal scrolling and hide the end of the logic in code reviews. When a line is too long, break it **after** a comma or **before** an operator, and indent the continuation by 8 spaces (double indent) so it is clearly a continuation, not a new statement.

**BAD**

```

String message = "Dear " + student.getFirstName() + " " + student.getLastName() + ", your
fee of Rs. " + fee.getAmount() + " is due on " + fee.getDueDate() + ". Please pay before
the deadline to avoid a late fee.";

```

**GOOD**

```

String message = "Dear " + student.getFullName()
    + ", your fee of Rs. " + fee.getAmount()
    + " is due on " + fee.getDueDate()
    + ". Please pay before the deadline to avoid a late fee.";

// Or better: use String.format / formatted text
String message = String.format(
    "Dear %s, your fee of Rs. %.2f is due on %s. Please pay before the deadline.",
    student.getFullName(), fee.getAmount(), fee.getDueDate());

```

### 3.5 Vertical spacing — blank lines group related ideas

Use **one blank line** to separate methods, and to separate "paragraphs" of logic inside a method (declare variables, then validate, then compute, then return). Never use more than one blank line in a row, and never leave a blank line directly after `{` or before `}`.

**BAD**

```

public double calculateNetSalary(Employee employee) {

```

**BAD**

```

double basic = employee.getBasic();
double hra = basic * HRA_RATE;
double da = basic * DA_RATE;
double gross = basic + hra + da;
double pf = basic * PF_RATE;
double tax = calculateTax(gross);
double net = gross - pf - tax;

return net;
}
public void printSlip(Employee employee) { ... }

```

**GOOD**

```

public double calculateNetSalary(Employee employee) {
    double basic = employee.getBasic();
    double hra = basic * HRA_RATE;
    double da = basic * DA_RATE;
    double gross = basic + hra + da;

    double pf = basic * PF_RATE;
    double tax = calculateTax(gross);

    return gross - pf - tax;
}

public void printSlip(Employee employee) { ... }

```

### 3.6 One statement per line, one variable per declaration

**BAD**

```

int a = 1, b = 2, c = 3; String name, email;
if (x > 0) y = 1; else y = -1;
count++; total += count;

```

**GOOD**

```

int minScore = 1;
int maxScore = 2;
int passScore = 3;
String name;
String email;

int sign = (x > 0) ? 1 : -1;

count++;
total += count;

```

### 3.7 Order of members inside a class

Readers expect a class to be laid out top-to-bottom in a predictable order. Follow this sequence:

1. Constants (`static final`)
2. Static fields
3. Instance fields (`private`)
4. Constructors
5. Public methods (the API of the class — what callers use)
6. Private helper methods (details, near the public method that uses them)
7. `equals`, `hashCode`, `toString`
8. Nested classes and enums

#### Well-ordered class

```
public class OtpService {

    private static final int OTP_LENGTH = 6;
    private static final int EXPIRY_MINUTES = 5;

    private final OtpRepository otpRepository;
    private final Clock clock;

    public OtpService(OtpRepository otpRepository, Clock clock) {
        this.otpRepository = otpRepository;
        this.clock = clock;
    }

    public String generateOtp(String mobileNumber) {
        String otp = randomDigits(OTP_LENGTH);
        otpRepository.save(mobileNumber, otp, expiryTime());
        return otp;
    }

    public boolean verifyOtp(String mobileNumber, String enteredOtp) {
        return otpRepository.matches(mobileNumber, enteredOtp, clock.instant());
    }

    private String randomDigits(int length) { ... }

    private Instant expiryTime() {
        return clock.instant().plus(EXPIRY_MINUTES, ChronoUnit.MINUTES);
    }
}
```

### 3.8 Imports

- Never use wildcard imports (`import java.util.*;`) — they hide which classes are used and can cause name clashes.
- Remove unused imports (your IDE greys them out; `Ctrl+Alt+0` in IntelliJ).
- Order: static imports first, then all other imports, alphabetically. Let the IDE do it.

#### Practice exercises

1. Reformat this by hand, then compare with your IDE formatter: `public class T{public static`

```
void main(String[] a){int s=0;for(int i=1;i<=10;i++){if(i%2==0)s+=i;}System.out.println(s);}}
```

2. Find the bug caused by missing braces: `if (balance >= amount) balance -= amount; log("withdrawn"); else log("denied");` (Hint: it does not even compile — explain why.)

3. Take any program you wrote last semester and apply sections 3.1 to 3.7. Count how many lines exceeded 100 characters.

## 4. Comments and Documentation

Beginners are often told "comment everything". Professionals know the opposite is closer to the truth: **good code needs few comments**, because the names and structure already explain it. A comment is a confession that the code could not explain itself. Comments also rot — code gets changed, the comment above it does not, and now it actively lies to the reader.

### 4.1 Comment the WHY, not the WHAT

The code already says *\*what\** it does. A comment should add something the code cannot say: **why** this approach was chosen, a business rule, a link to a bug ticket, a warning about a non-obvious consequence.

#### BAD — comments that repeat the code

```
// increment i
i++;

// check if age is greater than 18
if (age > 18) {

// loop through all students
for (Student student : students) {

// set the name
this.name = name;
```

#### GOOD — comments that explain intent and context

```
// RBI rule: KYC is mandatory only for adults; minors use a guardian's KYC.
if (age > 18) {

// Sort before de-duplicating: the duplicate check relies on adjacent equality.
Collections.sort(entries);

// Retry once on timeout - the payment gateway is flaky during peak hours (ticket PAY-231).
if (response.isTimeout()) {
```

### 4.2 Javadoc for anything public

Every **public class** and **public method** that other people will call should have a Javadoc comment (`/** ... */`). It describes the purpose, the parameters, the return value and the exceptions. Tools generate HTML documentation from it, and IDEs show it as a tooltip when someone calls your method.

**GOOD — Javadoc on a public method**

```
/**
 * Calculates the EMI (equated monthly instalment) for a loan using
 * the standard reducing-balance formula.
 *
 * @param principal    loan amount in rupees; must be positive
 * @param annualRate   yearly interest rate as a percentage, e.g. 8.5
 * @param tenureMonths number of monthly instalments; must be at least 1
 * @return the monthly instalment, rounded to two decimal places
 * @throws IllegalArgumentException if any argument is zero or negative
 */
public static double calculateEmi(double principal, double annualRate, int tenureMonths) {
    ...
}
```

- Start with a one-sentence summary in the third person ("Calculates...", "Returns..."), not "This method calculates..."
- Document **units** (rupees, seconds, percentage) and **valid ranges** — that is what callers actually need.
- Private methods usually need no Javadoc if their name is clear; a short `//` comment is enough when it is not.

## 4.3 Comments that should never exist

**BAD**

```
// int oldTotal = calculateOldTotal();    <- commented-out code. Delete it; Git remembers.
// TODO: fix this later                   <- no owner, no ticket, never fixed
// Author: Ravi, Date: 12/03/2019         <- version control already tracks this
// ===== END OF METHOD =====        <- noise
/* This method was changed by Priya to fix the bug */ <- belongs in the commit message
// I don't know why this works but don't touch it    <- rewrite until you know
```

**GOOD — a TODO with an owner and a ticket**

```
// TODO(mahesh, ASSESS-142): replace with batch query once the DAO supports it.
```

## 4.4 Self-documenting code beats comments

Before writing a comment, ask: can I rename something or extract a method so the comment is unnecessary?

**BAD**

```
// check if the student is eligible for placement
if (s.getCgpa() >= 7.0 && s.getBacklogs() == 0 && s.getAttendance() >= 75) {
    ...
}
```

**GOOD — the comment became a method name**

```
if (isEligibleForPlacement(student)) {
    ...
}

private boolean isEligibleForPlacement(Student student) {
```

**GOOD — the comment became a method name**

```

return student.getCgpa() >= MIN_PLACEMENT_CGPA
    && student.getBacklogs() == 0
    && student.getAttendance() >= MIN_ATTENDANCE_PERCENT;
}

```

**Practice exercises**

1. Delete every comment from a program you wrote earlier. Read it again. Every place where you now feel lost is a place that needs a better name, not a comment.
2. Write a proper Javadoc block for a method `boolean isValidPan(String pan)` that checks an Indian PAN number format.
3. Classify each comment in section 4.3 as "delete", "move to commit message", or "convert to a ticket".

## 5. Writing Good Methods

A method is the basic unit of "doing". Most of the difference between beginner code and professional code shows up here: beginners write one giant `main` method; professionals write many small, well-named methods that read like a table of contents.

### 5.1 Small, and doing one thing

A method should do **one thing**, do it well, and do only it. A practical test: can you describe the method in one sentence without the word "and"? If not, split it. Aim for methods of **5–20 lines**; anything over 40 lines is a strong signal to break it up.

**BAD — input, calculation, grading and printing all mixed in one method**

```

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.print("Enter marks: ");
    int m1 = sc.nextInt(); int m2 = sc.nextInt(); int m3 = sc.nextInt();
    int total = m1 + m2 + m3;
    double avg = total / 3.0;
    String grade;
    if (avg >= 90) grade = "A";
    else if (avg >= 75) grade = "B";
    else if (avg >= 60) grade = "C";
    else grade = "F";
    System.out.println("Total: " + total);
    System.out.println("Average: " + avg);
    System.out.println("Grade: " + grade);
    if (grade.equals("F")) System.out.println("Please meet your mentor.");
}

```

**GOOD — main reads like a summary; each step is testable on its own**

```

public static void main(String[] args) {
    int[] marks = readMarks(3);
    double average = calculateAverage(marks);
    String grade = assignGrade(average);
    printReport(marks, average, grade);
}

```

**GOOD — main reads like a summary; each step is testable on its own**

```

}

private static int[] readMarks(int count) { ... }

private static double calculateAverage(int[] marks) {
    int total = 0;
    for (int mark : marks) {
        total += mark;
    }
    return (double) total / marks.length;
}

private static String assignGrade(double average) {
    if (average >= 90) {
        return "A";
    }
    if (average >= 75) {
        return "B";
    }
    if (average >= 60) {
        return "C";
    }
    return "F";
}

private static void printReport(int[] marks, double average, String grade) { ... }

```

## 5.2 Keep parameters few

Zero to two parameters is ideal; three is acceptable; four or more means the parameters probably belong together in an object. Long parameter lists are easy to call in the wrong order — especially when several have the same type.

**BAD**

```

void registerStudent(String firstName, String lastName, String email,
                    String phone, String college, int year, String branch,
                    boolean isHosteller) { ... }

// Call site - which String is which?
registerStudent("Asha", "Rao", "9876543210", "asha@x.com", "RVCE", 3, "CSE", true);

```

**GOOD**

```

void registerStudent(StudentRegistration registration) { ... }

// The object names every value; the compiler catches mistakes
StudentRegistration registration = new StudentRegistration.Builder()
    .firstName("Asha")
    .lastName("Rao")
    .email("asha@x.com")
    .phone("9876543210")
    .college("RVCE")
    .year(3)
    .branch("CSE")
    .hosteller(true)

```

**GOOD**

```

        .build();
    registerStudent(registration);

```

### 5.3 Guard clauses — return early, avoid deep nesting

When a method has preconditions, check them **at the top** and return or throw immediately. This keeps the "happy path" at the left margin, not buried five levels deep inside `if` blocks (the "arrow anti-pattern").

**BAD — the "arrow" shape; the real logic is hidden at the deepest level**

```

public double calculateDiscount(Customer customer, Order order) {
    double discount = 0;
    if (customer != null) {
        if (order != null) {
            if (order.getTotal() > 0) {
                if (customer.isPremium()) {
                    discount = order.getTotal() * 0.10;
                } else {
                    if (order.getTotal() > 5000) {
                        discount = order.getTotal() * 0.05;
                    }
                }
            }
        }
    }
    return discount;
}

```

**GOOD — preconditions first, then a flat happy path**

```

public double calculateDiscount(Customer customer, Order order) {
    Objects.requireNonNull(customer, "customer must not be null");
    Objects.requireNonNull(order, "order must not be null");
    if (order.getTotal() <= 0) {
        return 0;
    }

    if (customer.isPremium()) {
        return order.getTotal() * PREMIUM_DISCOUNT_RATE;
    }
    if (order.getTotal() > BULK_ORDER_THRESHOLD) {
        return order.getTotal() * BULK_DISCOUNT_RATE;
    }
    return 0;
}

```

### 5.4 No magic numbers or magic strings

A bare number or string literal in the middle of logic is a "magic" value: the reader has no idea what it means or where else it is used. Give it a named constant. The exception is genuinely obvious values such as `0`, `1`, `-1`, `"`, and `100` in a percentage.

**BAD**

```
if (password.length() < 8) { ... }
double total = price * 1.18;
if (status.equals("ACT")) { ... }
Thread.sleep(300000);
```

**GOOD**

```
private static final int MIN_PASSWORD_LENGTH = 8;
private static final double GST_MULTIPLIER = 1.18;
private static final long POLL_INTERVAL_MS = 5 * 60 * 1000;    // 5 minutes

if (password.length() < MIN_PASSWORD_LENGTH) { ... }
double totalWithGst = price * GST_MULTIPLIER;
if (status == AccountStatus.ACTIVE) { ... }    // enum instead of a string code
Thread.sleep(POLL_INTERVAL_MS);
```

## 5.5 Avoid boolean flag parameters

A true/false argument at a call site tells the reader nothing (`sendEmail(user, true)` — true what?). It also means the method does two different things. Split it into two clearly named methods, or use an enum.

**BAD**

```
void sendEmail(User user, boolean isWelcome) {
    if (isWelcome) { ... } else { ... }
}
sendEmail(user, true);    // unreadable at the call site
```

**GOOD**

```
void sendWelcomeEmail(User user) { ... }
void sendPasswordResetEmail(User user) { ... }

sendWelcomeEmail(user);
```

## 5.6 Return what the name promises; no side effects

A method whose name is a question (`isValid`) must not change anything. A method that returns a value should not also print, save, or mutate its parameters. Mixing "query" and "command" makes code impossible to reason about.

**BAD**

```
public boolean isValidEmail(String email) {
    if (email == null || !email.contains("@")) {
        System.out.println("Invalid email!");    // side effect: printing
        invalidCount++;                          // side effect: mutating state
        return false;
    }
    return true;
}
```

**GOOD**

```
public boolean isValidEmail(String email) {
    return email != null && EMAIL_PATTERN.matcher(email).matches();
}

// The caller decides what to do about it
if (!isValidEmail(email)) {
    invalidCount++;
    logger.warn("Invalid email received: {}", email);
}
```

**Practice exercises**

1. Take the BAD example from 5.1 and split it further: `readMarks` should validate that each mark is between 0 and 100 and ask again if not.
2. Refactor: `double price(double p, int q, boolean member, boolean festival)` — remove both boolean flags.
3. Rewrite this with guard clauses: `if (user != null) { if (user.isActive()) { if (user.hasRole("ADMIN")) { grantAccess(); } } }`.

## 6. Writing Good Classes

### 6.1 One responsibility per class

A class should have **one reason to change**. If your `Student` class also connects to the database, formats PDF reports and sends emails, then a change to the email provider forces you to edit (and risk breaking) the `Student` class. Split responsibilities into separate classes with clear names.

**BAD — a "God class" that does everything**

```
class Student {
    String name; double cgpa;
    void saveToDatabase() { ... }           // persistence
    void generatePdfReport() { ... }        // reporting
    void sendResultEmail() { ... }          // notification
    double calculateScholarship() { ... }   // business rule
}
```

**GOOD**

```
class Student { ... } // just the data and simple behaviour
class StudentRepository { void save(Student s) { ... } }
class ReportGenerator { byte[] generatePdf(Student s) { ... } }
class NotificationService { void sendResultEmail(Student s) { ... } }
class ScholarshipCalculator { double calculate(Student s) { ... } }
```

## 6.2 Encapsulation — fields are private

Fields are **always** `private`. Expose behaviour through methods, and expose data only when necessary through getters. This lets you validate changes, change the internal representation later, and see every place that modifies state.

### BAD

```
public class BankAccount {
    public double balance;    // anyone can do account.balance = -99999;
    public String accountNo;
}
```

### GOOD — state can only change through methods that enforce the rules

```
public class BankAccount {
    private final String accountNumber;
    private double balance;

    public BankAccount(String accountNumber, double openingBalance) {
        if (openingBalance < 0) {
            throw new IllegalArgumentException("Opening balance cannot be negative");
        }
        this.accountNumber = accountNumber;
        this.balance = openingBalance;
    }

    public void deposit(double amount) {
        requirePositive(amount);
        balance += amount;
    }

    public void withdraw(double amount) {
        requirePositive(amount);
        if (amount > balance) {
            throw new InsufficientBalanceException(accountNumber, balance, amount);
        }
        balance -= amount;
    }

    public double getBalance() {
        return balance;
    }

    private static void requirePositive(double amount) {
        if (amount <= 0) {
            throw new IllegalArgumentException("Amount must be positive: " + amount);
        }
    }
}
```

## 6.3 Prefer immutability and final

Mark fields `final` whenever they are assigned once in the constructor. Immutable objects cannot be corrupted, are safe to share between threads, and are far easier to reason about. For simple data carriers, use a **record** (Java 16+), which is immutable by default and generates `equals`, `hashCode` and `toString` for you.

**BAD**

```
class Point {
    int x, y;
    Point() {}
    void setX(int x) { this.x = x; }
    void setY(int y) { this.y = y; }
    // plus hand-written equals/hashCode/toString, easy to get wrong
}
```

**GOOD**

```
record Point(int x, int y) { }
```

// Usage

```
Point origin = new Point(0, 0);
System.out.println(origin);           // Point[x=0, y=0]
System.out.println(origin.x());       // 0
```

## 6.4 Constructors should produce a valid object

An object should never exist in a half-built state. Validate in the constructor and throw if the inputs are wrong — do not rely on callers remembering to call `init()` or a series of setters.

## 6.5 Always override equals, hashCode and toString together

- If you override `equals`, you **must** override `hashCode` (otherwise `HashMap` and `HashSet` break silently).
- Use `Objects.equals()` and `Objects.hash()` — or let your IDE generate them, or use a record.
- A `toString` that prints the important fields makes debugging and logs dramatically easier.

### Practice exercises

1. Split this class: `class OrderManager { void addItem(); double calculateTotal(); void saveToDb(); void printInvoice(); void emailInvoice(); }` into 3–4 classes with single responsibilities.
2. Convert the `Point` class from 6.3 into an immutable class **without** using `record` (private final fields, constructor, getters, `equals`, `hashCode`, `toString`).
3. Write a `Money` record with `amount` and `currency` whose constructor rejects negative amounts and null currency.

# 7. Types, Variables and Declarations

## 7.1 Choose the right type

| Data          | Use                                     | Never use                                | Why                                                                                |
|---------------|-----------------------------------------|------------------------------------------|------------------------------------------------------------------------------------|
| Money, prices | <code>BigDecimal</code> (or long paise) | <code>double</code> , <code>float</code> | <code>0.1 + 0.2 != 0.3</code> in binary floating point; rounding errors accumulate |

| Data                            | Use                                                                        | Never use                                         | Why                                                            |
|---------------------------------|----------------------------------------------------------------------------|---------------------------------------------------|----------------------------------------------------------------|
| Counts, indexes                 | <code>int</code>                                                           | <code>double</code> , <code>String</code>         | Whole numbers; <code>int</code> is fast and clear              |
| Large numbers (IDs, timestamps) | <code>long</code>                                                          | <code>int</code>                                  | <code>int</code> overflows at ~2.1 billion                     |
| Yes/No                          | <code>boolean</code>                                                       | <code>int</code> 0/1, <code>String</code> "Y"     | Only two states; compiler-enforced                             |
| Fixed set of options            | <code>enum</code>                                                          | <code>String</code> , <code>int</code> codes      | Typos become compile errors; <code>switch</code> is exhaustive |
| Dates and times                 | <code>LocalDate</code> , <code>LocalDateTime</code> , <code>Instant</code> | <code>String</code> , <code>java.util.Date</code> | Correct arithmetic, time zones, formatting                     |
| Text                            | <code>String</code>                                                        | <code>char[]</code> (except passwords)            | Immutable and safe                                             |
| Nothing-or-value return         | <code>Optional&lt;T&gt;</code>                                             | <code>null</code>                                 | Forces caller to handle absence                                |

**BAD**

```
double price = 19.99;
double total = price * 3;           // 59.970000000000001
String status = "active";           // "Active"? "ACTIVE"? typo risk
String dob = "04/04/2000";          // is it dd/MM or MM/dd?
int phoneNumber = 9876543210;        // compile error: too large for int
```

**GOOD**

```
BigDecimal price = new BigDecimal("19.99");
BigDecimal total = price.multiply(BigDecimal.valueOf(3)); // 59.97 exactly
AccountStatus status = AccountStatus.ACTIVE;
LocalDate dateOfBirth = LocalDate.of(2000, 4, 4);
String phoneNumber = "9876543210"; // phone numbers are text, not numbers
```

## 7.2 Declare close to first use, and initialise

Do not declare all variables at the top of a method (C-style). Declare each one right where it is first needed, and give it a value immediately. Smallest possible scope means the reader holds less in their head.

**BAD**

```
public void generateReport() {
    int count;
    double sum;
    String line;
    List<String> lines;
    // ... 30 lines later ...
    count = 0;
    sum = 0;
    lines = new ArrayList<>();
```

**GOOD**

```
public void generateReport() {
    List<String> lines = new ArrayList<>();
    // ... use lines ...

    int count = 0;
    double sum = 0;
    for (Score score : scores) { ... }
```

## 7.3 Program to the interface

Declare variables, fields and parameters with the **interface** type (`List`, `Map`, `Set`), and choose the implementation (`ArrayList`, `HashMap`) only on the right side of `new`. You can then swap implementations without touching any other line.

**BAD**

```
ArrayList<String> names = new ArrayList<>();
HashMap<String, Integer> ages = new HashMap<>();
public ArrayList<Student> findAll() { ... }
```

**GOOD**

```
List<String> names = new ArrayList<>();
Map<String, Integer> agesByName = new HashMap<>();
public List<Student> findAll() { ... }
```

## 7.4 Using var sensibly

Java 10+ allows `var` for local variables. Use it only when the type is **obvious from the right-hand side**; otherwise the reader loses information.

**BAD**

```
var result = service.process(input);    // what type is result?
var x = 10;                             // int? long? unclear intent
```

**GOOD**

```
var students = new ArrayList<Student>();           // obvious
var reader = new BufferedReader(new FileReader(path));
Map<String, List<Order>> ordersByCustomer = groupOrders(orders); // explicit when helpful
```

## 7.5 Strings — the three classic mistakes

**BAD**

```
if (name == "admin") { ... }                // compares references, not content

String csv = "";
for (String item : items) {
    csv = csv + item + ",";                  // creates a new String every iteration
}
```

**BAD**

```
String sql = "SELECT * FROM users WHERE name = '" + userInput + "'"; // SQL injection
```

**GOOD**

```
if ("admin".equals(name)) { ... } // null-safe content comparison

String csv = String.join(",", items); // or a StringBuilder in a loop

PreparedStatement stmt = connection.prepareStatement(
    "SELECT * FROM users WHERE name = ?");
stmt.setString(1, userInput); // parameterised query
```

**Practice exercises**

1. A shopping cart stores prices as `double`. Add `0.1` ten times in a loop and print the result. Then do the same with `BigDecimal`. Explain the difference.
2. Rewrite `int day = 3; if (day == 3) { ... }` using an enum `DayOfWeek`. (Java already has one — find it.)
3. Find every `==` comparison of Strings in your last project and replace it with `equals`.

## 8. Control Flow: Conditions, Loops and Switch

### 8.1 Simplify boolean expressions

**BAD**

```
if (isValid == true) { ... }
if (isValid == false) { ... }
if (list.size() == 0) { ... }
if (x > 5) { return true; } else { return false; }
if (!(a > b)) { ... }
boolean isAdult = age >= 18 ? true : false;
```

**GOOD**

```
if (isValid) { ... }
if (!isValid) { ... }
if (list.isEmpty()) { ... }
return x > 5;
if (a <= b) { ... }
boolean isAdult = age >= 18;
```

### 8.2 Name complex conditions

**BAD**

```
if ((order.getTotal() > 1000 && customer.getTier() == Tier.GOLD)
```

**BAD**

```

    || (order.getItems().size() > 10 && !order.isInternational())
    || customer.hasCoupon("FEST24")) {
    applyDiscount(order);
}

```

**GOOD**

```

boolean isHighValueGoldOrder = order.getTotal() > 1000 && customer.getTier() == Tier.GOLD;
boolean isBulkDomesticOrder = order.getItems().size() > 10 && !order.isInternational();
boolean hasFestivalCoupon = customer.hasCoupon("FEST24");

if (isHighValueGoldOrder || isBulkDomesticOrder || hasFestivalCoupon) {
    applyDiscount(order);
}

```

## 8.3 Loops — prefer the enhanced for loop and streams

Use the classic `for (int i = 0; ...)` only when you actually need the index. For "do something to each element", use the enhanced for loop. For "transform / filter / sum", streams are often clearer once you know them.

**BAD**

```

for (int i = 0; i < students.size(); i++) {
    Student s = students.get(i);
    if (s.getCgpa() >= 8.0) {
        toppers.add(s.getName());
    }
}

```

**GOOD**

```

for (Student student : students) {
    if (student.getCgpa() >= TOPPER_CGPA) {
        toppers.add(student.getName());
    }
}

// Or with streams:
List<String> toppers = students.stream()
    .filter(student -> student.getCgpa() >= TOPPER_CGPA)
    .map(Student::getName)
    .toList();

```

## 8.4 Switch — use the modern arrow form

Old-style `switch` with `break` is a well-known source of "fall-through" bugs when a `break` is forgotten. Java 14+ has the arrow form, which cannot fall through and can return a value.

**BAD**

```

String label;
switch (day) {
    case MONDAY:

```

**BAD**

```

case TUESDAY:
    label = "Weekday";
    break;
case SATURDAY:
    label = "Weekend";           // forgot break - falls into default!
default:
    label = "Unknown";
}

```

**GOOD**

```

String label = switch (day) {
    case MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY -> "Weekday";
    case SATURDAY, SUNDAY -> "Weekend";
}; // compiler verifies every enum value is covered

```

## 8.5 Avoid long if-else chains on type or status

An if-else chain that checks `type == X`, `type == Y`, `type == Z` will grow forever. Prefer an enum with behaviour, a switch expression, or polymorphism (one class per type).

### Practice exercises

1. Simplify: `if (user.isActive() == true && user.isBlocked() == false) { return true; } else { return false; }`
2. Convert a classic switch with break from any old program to the arrow form.
3. Rewrite a loop that sums the even numbers in a list using a stream.

## 9. Error Handling

Programs fail. Files go missing, networks drop, users type "abc" where a number was expected. The difference between a hobby program and a professional one is not that the professional one never fails — it is that it fails **clearly, safely and early**, and tells someone what went wrong.

### 9.1 Exceptions in 60 seconds

- An **exception** is an object that represents a problem. When code cannot continue, it **throws** one; the nearest enclosing **catch** that matches handles it. If nothing catches it, the program stops and prints a stack trace.
- **Unchecked** exceptions (`RuntimeException` and subclasses such as `IllegalArgumentException`, `NullPointerException`, `IndexOutOfBoundsException`) signal **programming mistakes** or invalid arguments. You do not have to declare them.
- **Checked** exceptions (`IOException`, `SQLException`) signal **external problems** the caller may be able to recover from. The compiler forces you to catch or declare them with **throws**.
- Modern practice: use unchecked exceptions for most application errors, and reserve checked ones for cases where the caller really is expected to recover.

## 9.2 Never swallow an exception

An empty `catch` block is the single worst thing you can write in Java. The program keeps running in a broken state, and the real cause is lost forever. If you catch it, you must **handle it, log it, or rethrow it**.

### BAD

```
try {
    saveOrder(order);
} catch (Exception e) {
    // nothing here - the order silently vanished
}

try {
    int age = Integer.parseInt(input);
} catch (Exception e) {
    e.printStackTrace();    // "handled" by printing; program continues with age = 0?
}
```

### GOOD

```
try {
    saveOrder(order);
} catch (DatabaseException e) {
    logger.error("Failed to save order {}", order.getId(), e);
    throw new OrderProcessingException("Could not save order " + order.getId(), e);
}

// Or, when you can genuinely recover:
int age;
try {
    age = Integer.parseInt(input);
} catch (NumberFormatException e) {
    System.out.println("'" + input + "' is not a valid age. Please enter a number.");
    return;
}
```

## 9.3 Catch the most specific exception, not Exception or Throwable

Catching `Exception` also catches `NullPointerException` and every bug you did not anticipate, hiding them. Catch only what you can actually handle; let the rest propagate.

### BAD

```
try {
    readConfigFile(path);
} catch (Exception e) {           // also hides NPE, ClassCast, etc.
    useDefaults();
}
```

### GOOD

```
try {
    readConfigFile(path);
} catch (FileNotFoundException e) {
    logger.warn("Config file {} not found, using defaults", path);
    useDefaults();
} catch (IOException e) {
```

**GOOD**

```
throw new ConfigurationException("Could not read config file " + path, e);
}
```

## 9.4 Fail fast: validate inputs at the boundary

Check arguments at the **start** of a public method and throw `IllegalArgumentException` (or `NullPointerException` via `Objects.requireNonNull`) with a message that says what was wrong and what the value was. A bug caught at the entrance is easy to find; the same bug caught 200 lines later, as a mysterious `NullPointerException`, costs hours.

**BAD**

```
public double calculateEmi(double principal, double rate, int months) {
    return principal * rate / months;    // months = 0 -> Infinity; negative principal ->
nonsense
}
```

**GOOD**

```
public double calculateEmi(double principal, double annualRatePercent, int tenureMonths) {
    if (principal <= 0) {
        throw new IllegalArgumentException("principal must be positive, was " + principal);
    }
    if (annualRatePercent < 0) {
        throw new IllegalArgumentException("rate cannot be negative, was " +
annualRatePercent);
    }
    if (tenureMonths < 1) {
        throw new IllegalArgumentException("tenure must be at least 1 month, was " +
tenureMonths);
    }
    ...
}
```

## 9.5 Exception messages must be useful

**BAD**

```
throw new RuntimeException("Error");
throw new IllegalStateException("Something went wrong");
throw new Exception("invalid");
```

**GOOD**

```
throw new InsufficientBalanceException(
    "Cannot withdraw Rs. " + amount + " from account " + accountNumber
    + ": available balance is Rs. " + balance);

throw new IllegalArgumentException(
    "Expected roll number in format 1RV20CS001 but got '" + rollNumber + "'");
```

A good message contains **what was attempted, what value was involved, and why it failed**. Imagine reading it in a log file at midnight with no other context.

## 9.6 try-with-resources for anything that must be closed

Files, database connections, network sockets and streams must be closed even when an exception occurs. Never do this by hand with `finally`; use **try-with-resources**, which closes everything automatically in the right order.

### BAD

```
BufferedReader reader = null;
try {
    reader = new BufferedReader(new FileReader("students.csv"));
    String line = reader.readLine();
    ...
} catch (IOException e) {
    e.printStackTrace();
} finally {
    if (reader != null) {
        try { reader.close(); } catch (IOException ignored) { }
    }
}
```

### GOOD

```
try (BufferedReader reader = new BufferedReader(new FileReader("students.csv"))) {
    String line = reader.readLine();
    ...
} catch (IOException e) {
    throw new ImportException("Could not read students.csv", e);
}
```

## 9.7 Custom exceptions: name the business problem

When the standard exceptions do not describe the situation, create your own. Name it after the **business condition**, end it with `Exception`, extend `RuntimeException`, and always provide a constructor that accepts a cause so the original stack trace is preserved.

### GOOD

```
public class InsufficientBalanceException extends RuntimeException {

    public InsufficientBalanceException(String message) {
        super(message);
    }

    public InsufficientBalanceException(String message, Throwable cause) {
        super(message, cause); // never lose the original cause
    }
}
```

## 9.8 Do not use exceptions for normal control flow

### BAD

```
try {
    return list.get(index);
} catch (IndexOutOfBoundsException e) {
```

**BAD**

```

    return null;                // exceptions are slow and this hides intent
}

try {
    Integer.parseInt(text);
    return true;
} catch (NumberFormatException e) {
    return false;
}

```

**GOOD**

```

if (index < 0 || index >= list.size()) {
    return Optional.empty();
}
return Optional.of(list.get(index));

// For "is this a number?", check first or use a dedicated method
return text != null && text.matches("-?\\d+");

```

## 9.9 Logging instead of System.out

`System.out.println` is fine for classroom programs. In real applications, use a logging framework (SLF4J with Logback or Log4j2). Logs have **levels** (DEBUG, INFO, WARN, ERROR), timestamps, and can be switched on and off without changing code. Never log passwords, OTPs, card numbers or Aadhaar numbers.

**GOOD**

```

private static final Logger logger = LoggerFactory.getLogger(PaymentService.class);

logger.info("Processing payment {} for order {}", paymentId, orderId);
logger.warn("Retrying payment {} after timeout (attempt {})", paymentId, attempt);
logger.error("Payment {} failed permanently", paymentId, exception); // exception last =
stack trace

```

**Practice exercises**

1. Write `int readPositiveInt(Scanner scanner)` that keeps asking until the user enters a valid positive integer, handling `InputMismatchException` without crashing.
2. Create `StudentNotFoundException` and use it in `Student findByRollNumber(String rollNumber)`. Include the roll number in the message.
3. Rewrite a file-reading method you have written before using try-with-resources.
4. Find an empty catch block in any code you have (or online) and explain what bug it could hide.

## 10. Edge Case Handling

An **edge case** is an input at the boundary of what your code expects: empty, null, zero, negative, maximum, duplicates, unusual characters. Beginners test with "normal" values (5, "Asha", a list of three items) and are

then surprised in production. Professionals ask, for every input, "**what is the strangest legal value this could be?**" — and test that first.

## 10.1 The edge-case checklist

Run through this list for every method that accepts input:

| Input type           | Edge cases to consider                                                                                                                                                                                                                                                 |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Numbers              | 0, 1, -1, negative, very large (overflow), <code>Integer.MAX_VALUE</code> , decimals where an integer is expected, <code>NaN / Infinity</code> for doubles                                                                                                             |
| Strings              | <code>null</code> , empty <code>" "</code> , only spaces <code>"   "</code> , very long, leading/trailing spaces, mixed case, Unicode (names like "Müller", Kannada text), special characters ( <code>'</code> , <code>"</code> , <code>&lt;</code> , <code>;</code> ) |
| Collections / arrays | <code>null</code> , empty, exactly one element, duplicates, already sorted, reverse sorted, contains <code>null</code> elements, very large                                                                                                                            |
| Indexes / ranges     | first element (0), last element ( <code>length - 1</code> ), one past the end ( <code>length</code> ), negative, <code>start &gt; end</code> , <code>start == end</code>                                                                                               |
| Dates                | leap year (29 Feb), month/year boundaries, time zones, dates in the past/future, today                                                                                                                                                                                 |
| Objects              | <code>null</code> , partially filled (some fields <code>null</code> ), the same object passed twice                                                                                                                                                                    |
| Money                | zero, negative, more than balance, rounding to paise, currency mismatch                                                                                                                                                                                                |
| Files / network      | missing, empty, no permission, disconnected mid-way, slow                                                                                                                                                                                                              |

## 10.2 Null — the billion-dollar mistake

`NullPointerException` is the most common crash in Java. Defend against it in three ways: **never return null** from your own methods (return an empty collection or an `Optional`), **validate parameters** at the boundary, and **check external data** (database rows, API responses, user input) before using it.

### BAD

```
public List<Order> findOrders(String customerId) {
    if (customerId == null) return null;           // caller now has to remember to check
    ...
}

public Student findTopper(List<Student> students) {
    Student topper = null;                        // returns null when the list is empty
    for (Student s : students) { ... }
    return topper;
}

// Caller crashes:
String name = findTopper(students).getName();    // NPE if list was empty
```

### GOOD

```
public List<Order> findOrders(String customerId) {
    Objects.requireNonNull(customerId, "customerId must not be null");
    ...
    return orders.isEmpty() ? List.of() : orders; // empty list, never null
}

public Optional<Student> findTopper(List<Student> students) {
```

**GOOD**

```

    return students.stream().max(Comparator.comparingDouble(Student::getCgpa));
}

// Caller is forced to think about the empty case:
String name = findTopper(students)
    .map(Student::getName)
    .orElse("No students enrolled");

```

## 10.3 Empty and blank input

**BAD**

```

public double average(int[] marks) {
    int sum = 0;
    for (int m : marks) sum += m;
    return sum / marks.length;           // ArithmeticException when marks is empty
}

if (name.equals("")) { ... }           // NPE if name is null; misses " "

```

**GOOD**

```

public double average(int[] marks) {
    if (marks == null || marks.length == 0) {
        throw new IllegalArgumentException("marks must contain at least one value");
    }
    int sum = 0;
    for (int mark : marks) {
        sum += mark;
    }
    return (double) sum / marks.length; // (double) avoids integer division
}

if (name == null || name.isBlank()) { ... } // isBlank() handles " " too

```

## 10.4 Boundaries and off-by-one errors

The most common loop bug is being off by one at the end. Always ask: is the last valid index `length - 1`? Is the range inclusive or exclusive? Test with an array of size 0, 1 and 2.

**BAD**

```

for (int i = 0; i <= arr.length; i++) { // <= runs one step too far
    System.out.println(arr[i]);           // ArrayIndexOutOfBoundsException
}

String last = name.substring(name.length()); // "" - probably wanted the last char
char lastChar = name.charAt(name.length()); // StringIndexOutOfBoundsException

```

**GOOD**

```

for (int i = 0; i < arr.length; i++) {
    System.out.println(arr[i]);
}

```

**GOOD**

```

}

char lastChar = name.charAt(name.length() - 1);    // after checking name is not empty

```

## 10.5 Integer overflow and division

**BAD**

```

int seconds = days * 24 * 60 * 60;           // overflows for days > 24,855
int average = (a + b) / 2;                   // a + b can overflow for large a, b
int percentage = marks / total * 100;        // integer division: 45 / 50 = 0, then 0 * 100 = 0
double ratio = count / size;                 // same problem

```

**GOOD**

```

long seconds = days * 24L * 60 * 60;         // long arithmetic from the first multiply
int average = a + (b - a) / 2;                // overflow-safe midpoint
int percentage = marks * 100 / total;         // multiply first, then divide
double ratio = (double) count / size;        // cast before dividing

// When you must detect overflow:
int product = Math.multiplyExact(quantity, unitPrice);    // throws ArithmeticException

```

## 10.6 Floating point comparison

**BAD**

```

double total = 0.1 + 0.2;
if (total == 0.3) { ... }                // false! total is 0.30000000000000004

```

**GOOD**

```

private static final double EPSILON = 1e-9;

if (Math.abs(total - 0.3) < EPSILON) { ... }

// For money, avoid double entirely:
BigDecimal total = new BigDecimal("0.10").add(new BigDecimal("0.20"));
if (total.compareTo(new BigDecimal("0.30")) == 0) { ... }

```

## 10.7 Modifying a collection while iterating

**BAD**

```

for (Student student : students) {
    if (student.hasGraduated()) {
        students.remove(student);           // ConcurrentModificationException
    }
}

```

**GOOD**

```
students.removeIf(Student::hasGraduated);

// Or with an explicit iterator when you need more control:
Iterator<Student> iterator = students.iterator();
while (iterator.hasNext()) {
    if (iterator.next().hasGraduated()) {
        iterator.remove();
    }
}
```

## 10.8 Untrusted input from users and files

- **Trim and normalise** text before comparing: `input.trim().toLowerCase()`.
- **Parse, then validate:** `Integer.parseInt` throws on "12a"; a negative age parses fine but is still invalid — check both.
- **Never trust the format:** a CSV row may have fewer columns than expected; check `parts.length` before `parts[3]`.
- **Limit sizes:** a file upload of 5 GB or a name of 10,000 characters is an attack or a mistake — reject early.

### Write the edge-case tests first

Before writing the method body, write down 3–5 strange inputs and what should happen for each. Then write the method so that all of them behave correctly. This is the heart of test-driven thinking, and interviewers at product companies specifically look for it: when given a problem, the strong candidate immediately asks "What if the array is empty? Can the values be negative?"

### Practice exercises

1. Write `int findMax(int[] values)`. List every edge case first, decide what each should do (throw? return a sentinel?), then implement it.
2. Write `String getInitials(String fullName)` for inputs such as "Mahesh Arali", " asha", "A", "", null, "jean-paul sartre". Decide and document the behaviour for each.
3. Write `double safePercentage(int part, int total)` that never divides by zero and never suffers integer division.
4. Take a method from an earlier assignment and pass it an empty list, a single-element list, and null. Fix what breaks.

## 11. Code Length, Duplication and Structure

### 11.1 How long is too long?

| Unit   | Comfortable          | Warning sign                        | What to do                                               |
|--------|----------------------|-------------------------------------|----------------------------------------------------------|
| Line   | up to 100 characters | over 100                            | Wrap after commas / before operators; extract a variable |
| Method | 5–20 lines           | over 40 lines, or needs a scrollbar | Extract helper methods; each one gets a name             |

| Unit          | Comfortable      | Warning sign              | What to do                                           |
|---------------|------------------|---------------------------|------------------------------------------------------|
| Parameters    | 0–3              | 4 or more                 | Group into a parameter object / record               |
| Class         | under 300 lines  | over 500 lines            | Split by responsibility                              |
| Nesting depth | 1–2 levels       | 3 or more                 | Guard clauses; extract the inner block into a method |
| File          | one public class | several unrelated classes | One top-level class per file                         |

These numbers are guidelines, not laws. The real test is whether a reader can hold the whole unit in their head at once.

## 11.2 Don't Repeat Yourself (DRY)

When you copy-paste a block and change one thing, you have created two places that must be fixed every time the logic changes — and one of them will be forgotten. Extract the shared logic into a method with parameters for the differences.

### BAD

```
double mathPercent = (double) mathMarks / MAX_MARKS * 100;
if (mathPercent < 0 || mathPercent > 100) throw new IllegalStateException("bad maths");
System.out.printf("Maths: %.1f%%\n", mathPercent);

double sciencePercent = (double) scienceMarks / MAX_MARKS * 100;
if (sciencePercent < 0 || sciencePercent > 100) throw new IllegalStateException("bad sci");
System.out.printf("Science: %.1f%%\n", sciencePercent);

double englishPercent = (double) englishMarks / MAX_MARKS * 100;
if (englishPercent < 0 || englishPercent > 100) throw new IllegalStateException("bad eng");
System.out.printf("English: %.1f%%\n", englishPercent);
```

### GOOD

```
printPercentage("Maths", mathMarks);
printPercentage("Science", scienceMarks);
printPercentage("English", englishMarks);

private void printPercentage(String subject, int marks) {
    double percent = toPercentage(marks);
    System.out.printf("%s: %.1f%%\n", subject, percent);
}

private double toPercentage(int marks) {
    if (marks < 0 || marks > MAX_MARKS) {
        throw new IllegalArgumentException("marks out of range: " + marks);
    }
    return (double) marks / MAX_MARKS * 100;
}
```

## 11.3 Delete dead code

Unused variables, unused methods, unreachable branches and commented-out blocks are all **dead code**. They confuse readers ("is this important?") and hide in searches. Delete them — version control keeps the history if you ever need it back.

## 11.4 Keep related things together

- A private helper method goes **directly below** the public method that first uses it.
- Declare a variable **just before** the first line that uses it, not at the top of the method.
- Keep a class's fields, constructor and methods that use those fields in the same class — do not scatter one concept across five files.

## 12. Putting It All Together: A Complete Before-and-After

Here is a typical first-year program: a student result calculator. It works. It also breaks almost every rule in this document. Read the BAD version first and count the problems you can spot before reading the list.

### BAD — the "it runs, so it is done" version

```
import java.util.*;
public class prog1 {
public static void main(String[] args) {
Scanner sc=new Scanner(System.in);
String n=sc.nextLine();
int a=sc.nextInt();int b=sc.nextInt();int c=sc.nextInt();
int t=a+b+c;
double p=t/300*100;
String g;
if(p>=90) g="A";
else if(p>=75) g="B";
else if(p>=60) g="C";
else if(p>=40) g="D";
else g="F";
System.out.println(n+" "+t+" "+p+" "+g);
// System.out.println("debug "+a);
if(g=="F") System.out.println("fail");
}
}
```

Problems, in order of appearance: class name is lowercase and meaningless; wildcard import; no indentation; single-letter variables (n, a, b, c, t, p, g); no spaces around operators; no braces on `if`; `t/300*100` is integer division, so `p` is always 0 — **a real bug caused by unclear code**; magic numbers 300, 90, 75, 60, 40; no validation (marks of 150 or -20 are accepted; non-numeric input crashes with a stack trace); commented-out debug line; `g == "F"` compares references, not content; everything in one method; output is unlabelled.

### GOOD — the professional version

```
import java.util.Scanner;

/**
 * Reads a student's marks in three subjects and prints the total,
 * percentage and letter grade.
 */
public class StudentResultCalculator {

    private static final int SUBJECT_COUNT = 3;
    private static final int MAX_MARKS_PER_SUBJECT = 100;
    private static final int MAX_TOTAL_MARKS = SUBJECT_COUNT * MAX_MARKS_PER_SUBJECT;

    public static void main(String[] args) {
        try (Scanner scanner = new Scanner(System.in)) {
```

**GOOD — the professional version**

```

        System.out.print("Enter student name: ");
        String studentName = readNonBlankLine(scanner);

        int[] marks = new int[SUBJECT_COUNT];
        for (int i = 0; i < SUBJECT_COUNT; i++) {
            marks[i] = readMarks(scanner, "Enter marks for subject " + (i + 1) + ": ");
        }

        int totalMarks = sum(marks);
        double percentage = calculatePercentage(totalMarks);
        String grade = assignGrade(percentage);

        printResult(studentName, totalMarks, percentage, grade);
    }
}

private static String readNonBlankLine(Scanner scanner) {
    while (true) {
        String line = scanner.nextLine().trim();
        if (!line.isBlank()) {
            return line;
        }
        System.out.print("Name cannot be empty. Please enter again: ");
    }
}

private static int readMarks(Scanner scanner, String prompt) {
    while (true) {
        System.out.print(prompt);
        try {
            int marks = Integer.parseInt(scanner.nextLine().trim());
            if (marks >= 0 && marks <= MAX_MARKS_PER_SUBJECT) {
                return marks;
            }
            System.out.println("Marks must be 0 to " + MAX_MARKS_PER_SUBJECT + ".");
        } catch (NumberFormatException e) {
            System.out.println("Please enter a whole number.");
        }
    }
}

private static int sum(int[] values) {
    int total = 0;
    for (int value : values) {
        total += value;
    }
    return total;
}

private static double calculatePercentage(int totalMarks) {
    return (double) totalMarks / MAX_TOTAL_MARKS * 100;
}

private static String assignGrade(double percentage) {
    if (percentage >= 90) {
        return "A";
    }
    if (percentage >= 75) {

```

**GOOD — the professional version**

```

        return "B";
    }
    if (percentage >= 60) {
        return "C";
    }
    if (percentage >= 40) {
        return "D";
    }
    return "F";
}

private static void printResult(String name, int total, double percent, String grade) {
    System.out.printf("%nStudent : %s%n", name);
    System.out.printf("Total      : %d / %d%n", total, MAX_TOTAL_MARKS);
    System.out.printf("Percent   : %.2f%%%n", percent);
    System.out.printf("Grade     : %s%n", grade);
    if ("F".equals(grade)) {
        System.out.println("Result      : FAIL - please meet your mentor.");
    }
}
}

```

The good version is longer — roughly four times as many lines. That is normal and expected. In return it has **no bugs**, survives any input a user can type, and every method can be understood and tested on its own. A reviewer can read `main` in ten seconds and know exactly what the program does.

**Practice exercises**

1. Extend the GOOD version to accept any number of subjects (ask the user how many first). Which methods change? Which stay the same? That stability is the payoff of good structure.
2. Extract the grading rules (90, 75, 60, 40) into an enum `Grade` with a static method `Grade.fromPercentage(double)`.
3. Write three unit tests for `assignGrade`: a boundary value (exactly 75), a value just below it (74.99), and an impossible value (105). Decide what should happen for the last one.

## 13. Self-Review Checklist

Run through this list before submitting any assignment, and later before raising any pull request. Reviewers at companies check exactly these things. If you can tick every box, your code will look professional.

### Naming

- ☐ Every variable, method and class name reveals its purpose without a comment.
- ☐ No single-letter names except loop counters and lambda parameters.
- ☐ No abbreviations that a new teammate would not recognise.
- ☐ Classes are PascalCase nouns; methods are camelCase verbs; constants are UPPER\_SNAKE\_CASE.
- ☐ Booleans read as questions (`isX`, `hasX`, `canX`).
- ☐ Collections are plural nouns.

## Formatting

- ☐ Code was run through the IDE formatter; 4-space indentation, no tabs.
- ☐ Every `if`, `for`, `while` has braces.
- ☐ Opening braces at the end of the line; closing braces on their own line.
- ☐ Spaces around operators and after commas; no line over 100 characters.
- ☐ One statement per line; one variable per declaration.
- ☐ No wildcard or unused imports.

## Methods and classes

- ☐ Every method does one thing and fits on a screen (under ~20 lines).
- ☐ No method takes more than three parameters; no boolean flag parameters.
- ☐ Guard clauses at the top; nesting never deeper than two levels.
- ☐ No magic numbers or strings — all named constants or enums.
- ☐ Fields are private; objects are valid immediately after construction.
- ☐ No duplicated blocks; no dead or commented-out code.

## Comments

- ☐ Comments explain *\*why\**, never *\*what\**.
- ☐ Every public class and method has a Javadoc summary.
- ☐ No commented-out code, no author/date comments, no TODOs without an owner.

## Error handling

- ☐ No empty `catch` blocks; no `catch (Exception e)` unless rethrowing.
- ☐ Public methods validate their arguments and throw with descriptive messages.
- ☐ All resources (files, connections, scanners) use try-with-resources.
- ☐ Exceptions carry the original cause (`new RuntimeException(message, e)`).
- ☐ No exceptions used for normal control flow.

## Edge cases

- ☐ Tested with empty, null, single-element and very large inputs.
- ☐ No method of mine returns `null` — empty collection or `Optional` instead.
- ☐ No integer division where a decimal is expected; no `==` on Strings or doubles.
- ☐ Loops checked for off-by-one at both ends.
- ☐ Money uses `BigDecimal`; dates use `java.time`.

## 14. Appendix: Tools and Further Reading

### 14.1 Let the tools enforce the rules

| Tool           | What it does                                                               | How to use it                                                                                                                                                                   |
|----------------|----------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| IDE formatter  | Fixes indentation, spacing, braces, imports                                | IntelliJ: <code>Ctrl+Alt+L</code> ; Eclipse: <code>Ctrl+Shift+F</code> ; VS Code: <code>Shift+Alt+F</code> . Import the Google Java Style XML so everyone formats the same way. |
| Checkstyle     | Flags naming and formatting violations at build time                       | Add the Maven/Gradle plugin with <code>google_checks.xml</code> . Fails the build on violations.                                                                                |
| SpotBugs / PMD | Finds real bugs: empty catch, <code>==</code> on strings, unused variables | Maven/Gradle plugin; run before every commit.                                                                                                                                   |
| SonarLint      | Live code-quality hints inside the IDE                                     | Free IDE plugin; underlines problems as you type.                                                                                                                               |
| JUnit 5        | Unit tests — the only proof that edge cases work                           | One test class per production class; test names describe the scenario:<br><code>withdraw_throwsWhenAmountExceedsBalance()</code> .                                              |
| EditorConfig   | Shares indentation/line-length settings across editors                     | Put a <code>.editorconfig</code> file at the repository root.                                                                                                                   |

### 14.2 Recommended reading

- **Google Java Style Guide** — [google.github.io/styleguide/javaguide.html](https://google.github.io/styleguide/javaguide.html). The de-facto industry reference; short enough to read in an evening.
- **Oracle Code Conventions for the Java Programming Language** — the original 1999 document; still the basis for most company standards.
- **Clean Code** by Robert C. Martin — chapters 2 (Meaningful Names), 3 (Functions) and 7 (Error Handling) map directly onto this document.
- **Effective Java** (3rd edition) by Joshua Bloch — the book every Java developer is expected to have read; items on immutability, equals/hashCode, Optional and exceptions are essential.
- **Java documentation** — [docs.oracle.com/en/java/](https://docs.oracle.com/en/java/). Learn to read the official Javadoc for `String`, `List`, `Map`, `Optional` and `java.time`.

### 14.3 A note on interviews

In coding interviews, the interviewer is watching how you write, not only whether the answer runs. Naming variables clearly, asking about edge cases before coding, validating input, and structuring the solution into small methods are all scored explicitly at most product companies. Every habit in this document is also an interview skill.

#### Final thought

Beautiful code is not decoration. It is the cheapest form of documentation, the fastest form of onboarding, and the strongest defence against bugs. Write every line as if the person who has to maintain it is a tired colleague who knows where you live — and then make it easy for them.